
```
# Unpythonic Example
if 42 < spam and spam < 99:
```

But Python lets you chain comparison operators so you don't need to use the `and` operator. The following code is equivalent to the previous example:

```
# Pythonic Example
if 42 < spam < 99:
```

The same applies to chaining the `=` assignment operator. You can set multiple variables to the same value in a single line of code:

```
>>> # Pythonic Example
>>> spam = eggs = bacon = 'string'
>>> print(spam, eggs, bacon)
string string string
```

To check whether all three of these variables are the same, you can use the `and` operator, or more simply, chain the `==` comparison operator for equality.

```
>>> # Pythonic Example
>>> spam = eggs = bacon = 'string'
>>> spam == eggs == bacon == 'string'
True
```

Chaining operators is a small but useful shortcut in Python. However, if you use them incorrectly, they can cause problems. Chapter 8 demonstrates some instances where using them can introduce unexpected bugs in your code.

Checking Whether a Variable Is One of Many Values

You might sometimes encounter the inverse of the situation described in the preceding section: checking whether a single variable is one of multiple possible values. You could do this using the `or` operator, such as in the expression `spam == 'cat' or spam == 'dog' or spam == 'moose'`. All of those redundant “`spam ==`” parts make this expression a bit unwieldy.